

try to cache the static content as close as possible to the users so that the content servers don't have to serve the information that doesn't change much every time there is a request for it.

Content management systems

As the technology moved forward, computing moved from large mainframes to smaller PCs. And the devices just kept getting smaller — to mobile phones, smart devices such as smart watches and digital assistants.

A traditional CMS includes two major components – a content storage and creation/management system that allows a content creator to create, manage and modify the content and content types; and a content delivery system that builds the content and delivers it to the users when they request for content. The former provides the content storage and management, and the latter is the presentation layer that fetches the raw content, and applies styles and the template to compile the final layout and deliver it to users. It is also possible to add more functionality by way of plugins or addons that complement the content delivery with more features such as advertising.

This worked when PCs were the primary devices to consume content. Think of a blog, video blog or even a news site – a traditional CMS ensures that the content writers can push content, collaborate with each other and then, the content gets delivered to users in the right styles and formatting without the creators doing anything about it.

Pure content management

However, content consumption moved from large screens to smaller ones — think mobile phones and smart watches — and even smart devices and digital assistants that do not have a screen. Traditional CMSs needed to now also provide a way to deliver content on these new devices and form factors. This was tackled by adding support for responsive themes that can adapt to

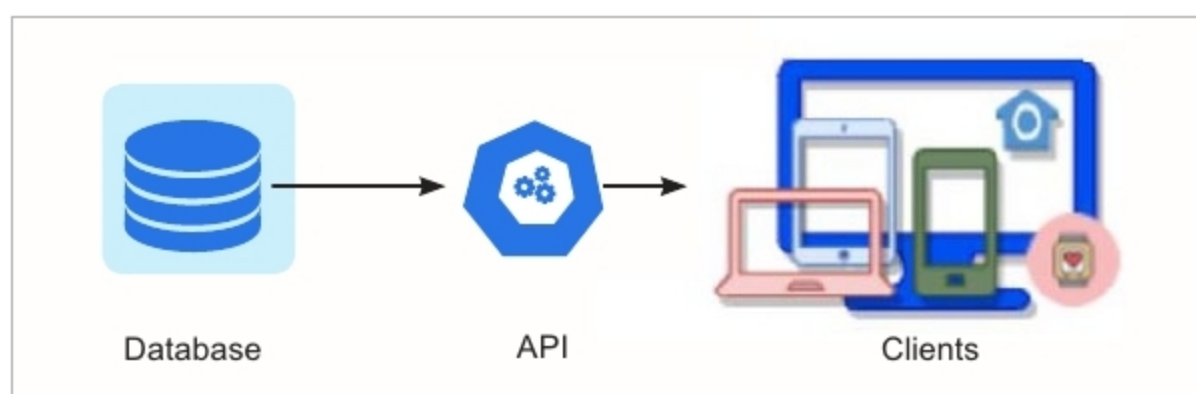


Figure 1: Basic design of a headless CMS

any screen size and layout. For devices without a screen, an API layer was added on top of the content storage engine. The apps that run on these devices can call these APIs and deliver the content as needed.

This works but it's just only a matter of 'when' rather than 'if', the computing and consumption is due for a change – just like smartphones arrived a little over a decade ago and changed our lives. We are not sure what this change will be and if you follow the innovations in hardware and technology, you are bound to have heard of things like transparent displays, foldable phones and even wall-sized panels.

That's where pure content management system comes into the picture – a system that can pave the way for any means of delivery on any kind of device with any size or type of display – even ones without a display! Open source developers (and others) have already done that and these systems are called headless CMSs – named so since they do not have a dependency, or a coupling with a presentation layer found in the traditional CMSs. Since headless CMSs store content in a structured format and without any notion about how the content will be consumed or presented to the users, they are considered 'pure'. Figure 1 illustrates a basic design of a headless CMS.

A number of popular headless CMSs are based on Jamstack (<https://jamstack.org/>), which stands for JavaScript, APIs and mark-up, and any software that doesn't have a tight

coupling between the server and client qualifies.

However, you will also find headless CMSs that use frameworks or languages other than JavaScript. Jamstack can be used on a variety of operating systems, Web servers, databases, back-ends or front-ends. It provides a higher degree of flexibility, extensibility and scale, as well as a great developer experience.

One thing is for sure — a headless CMS does not include a content-serving mechanism of any kind and that leads us to the following question.

How do I deliver content?

The traditional systems deliver content with their content delivery component. As a content creator, you are limited by their capabilities in this area. Additional engineering is needed to accommodate any other forms of content delivery that you may need. This means that you need to spend time and resources to ensure all your users can get the content in the same consistent way.

A headless CMS is meant to be designed API-first, and it exposes APIs for you to consume and serve the content from any medium.

Some of the traditional CMSs, like WordPress for example, also provide APIs to bridge this gap. But they are designed to be all-in-one solutions, so decoupling the parts you don't need is not possible. You can choose to ignore them though.

That's a Catch-22 situation – you either end up wasting computing resources on components you don't